

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

Институт компьютерных наук и технологического образования
Кафедра компьютерных технологий и электронного обучения

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения
и обработки больших данных»
форма обучения – очная

Курсовая работа

по дисциплине «Прикладные информационные технологии»
Разработка и развертывание ETL-конвейера для анализа дефектов
оборудования ТЭЦ с использованием Apache Airflow, Docker и ClickHouse

Обучающейся 3 курса
Блохина В. С.

Руководитель:
старший преподаватель Аксютин П. А.

Санкт-Петербург

2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПОСТРОЕНИЯ ETL-КОНВЕЙЕРОВ ДЛЯ АНАЛИЗА ДАННЫХ	6
1.1. Понятие ETL-процессов и их роль в аналитике данных	6
1.2. Анализ предметной области: дефекты оборудования ТЭЦ	7
1.3. Обзор современных технологий для построения ETL-конвейеров	9
1.4. Сравнительный анализ инструментов визуализации данных	11
1.5. Обоснование выбора технологического стека	12
Глава 2. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ ETL-КОНВЕЙЕРА	15
2.1. Общая архитектура решения	15
2.2. Контейнеризация с использованием Docker и Docker Compose	16
2.3. Реализация ETL-процессов в Apache Airflow	17
2.4. Настройка ClickHouse как аналитического хранилища	17
2.5. Визуализация данных в Apache Superset	18
Глава 3. ПРАКТИЧЕСКОЕ РАЗВЕРТЫВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ	21
3.1. Развертывание проекта	21
3.2. Загрузка и обработка данных о дефектах	22
3.3. Создание дашбордов для анализа дефектов ТЭЦ	22
3.4. Интерпретация полученных результатов	23
ЗАКЛЮЧЕНИЕ	25
СПИСОК ЛИТЕРАТУРЫ	27
ПРИЛОЖЕНИЕ А	28

ВВЕДЕНИЕ

Актуальность исследования. В современных условиях цифровой трансформации энергетического сектора особое значение приобретает автоматизация процессов сбора, обработки и анализа данных о состоянии оборудования. Теплоэлектроцентрали (ТЭЦ) являются критически важными объектами инфраструктуры, от бесперебойной работы которых зависит теплоснабжение и электроснабжение целых регионов. Дефекты оборудования ТЭЦ могут привести к серьезным авариям, экономическим потерям и угрозе для жизни людей.

Традиционные подходы к анализу дефектов, основанные на ручной обработке данных в электронных таблицах, не позволяют оперативно выявлять тенденции, прогнозировать отказы и принимать обоснованные управленческие решения. В связи с этим возникает необходимость внедрения современных ETL-решений (Extract, Transform, Load), которые обеспечивают автоматизированный сбор данных из различных источников, их трансформацию и загрузку в аналитические хранилища.

Особую актуальность приобретает использование контейнеризации (Docker) для развертывания аналитических систем, что позволяет стандартизировать среду выполнения, упростить масштабирование и обеспечить воспроизводимость результатов. Кроме того, применение открытых технологий (Airflow, ClickHouse, Superset) делает решение доступным для широкого круга организаций без значительных лицензионных отчислений.

Объект исследования – процессы сбора, обработки и анализа данных о дефектах оборудования теплоэлектроцентралей.

Предмет исследования – методы и инструменты построения ETL-конвейеров для анализа дефектов оборудования ТЭЦ.

Цель курсовой работы – разработка и развертывание ETL-конвейера для анализа дефектов оборудования ТЭЦ с использованием современных открытых

технологий (Apache Airflow, ClickHouse, Apache Superset) и контейнеризации Docker.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области – изучить типовые дефекты оборудования ТЭЦ и требования к их анализу.
2. Выполнить обзор и сравнительный анализ современных технологий для построения ETL-конвейеров и визуализации данных.
3. Обосновать выбор технологического стека для реализации проекта.
4. Разработать архитектуру ETL-конвейера.
5. Реализовать контейнеризацию всех компонентов с использованием Docker и Docker Compose.
6. Разработать DAG (Directed Acyclic Graph) в Apache Airflow для автоматизации ETL-процессов.
7. Настроить аналитическое хранилище ClickHouse.
8. Создать дашборды в Apache Superset для визуализации результатов анализа.
9. Составить документацию по развертыванию и использованию разработанного решения.

Методы исследования. В работе используются методы системного анализа, сравнительного анализа технологий, проектирования программных систем, контейнеризации, а также методы обработки и визуализации данных.

Научная новизна работы заключается в разработке комплексного ETL-решения для анализа дефектов оборудования ТЭЦ на основе открытых технологий с применением контейнеризации, что обеспечивает воспроизводимость и переносимость системы.

Практическая значимость состоит в том, что разработанное решение может быть использовано на предприятиях энергетического сектора для

автоматизации анализа дефектов оборудования, что позволит сократить время на обработку данных и повысить качество принимаемых решений.

Структура курсовой работы. Работа состоит из введения, трех глав, заключения, списка литературы и приложений. В первой главе рассматриваются теоретические основы построения ETL-конвейеров и проводится анализ предметной области. Во второй главе описывается архитектура решения и реализация всех компонентов. Третья глава посвящена практическому развертыванию и анализу полученных результатов.

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПОСТРОЕНИЯ ETL-КОНВЕЙЕРОВ ДЛЯ АНАЛИЗА ДАННЫХ

1.1. Понятие ETL-процессов и их роль в аналитике данных

ETL (Extract, Transform, Load) – это процесс, который лежит в основе современных систем управления данными и бизнес-аналитики. Термин ETL объединяет три ключевых этапа работы с данными:

Extract (Извлечение) – процесс получения данных из различных источников. Источниками могут быть реляционные базы данных, файлы (CSV, Excel, JSON), API-сервисы, системы мониторинга, журналы событий и т.д. На этом этапе данные собираются в исходном виде без какой-либо обработки.

Transform (Трансформация) – этап очистки, нормализации, обогащения и преобразования данных в формат, пригодный для анализа. Трансформация включает в себя:

- удаление дубликатов и пропущенных значений;
- приведение данных к единым форматам (дат, валют, единиц измерения);
- агрегацию данных (расчет сумм, средних значений, группировку);
- обогащение данных из справочников;
- применение бизнес-правил и логики.

Load (Загрузка) – финальный этап, на котором преобразованные данные загружаются в целевое хранилище – Data Warehouse, Data Lake или аналитическую базу данных. Загруженные данные становятся доступны для последующего анализа, построения отчетов и дашбордов.

Роль ETL-процессов в современной аналитике данных трудно переоценить. Согласно исследованию компании Gartner, до 80% времени аналитиков тратится на подготовку данных, а не на сам анализ. Автоматизация ETL-процессов позволяет:

- сократить время на подготовку данных с часов до минут;
- обеспечить актуальность аналитических отчетов;
- минимизировать человеческие ошибки при обработке данных;
- создать единый источник достоверных данных в организации.

Существует несколько архитектурных подходов к реализации ETL-процессов, они представлены в таблице 1.

Таблица 1. Архитектурные подходы к реализации ETL-процессов

Подход	Описание	Преимущества	Недостатки
Пакетная обработка (Batch)	Данные обрабатываются периодически (ежечасно, ежедневно)	Простота реализации, предсказуемая нагрузка	Задержка в актуальности данных
Streaming (потокковая)	Данные обрабатываются в реальном времени	Минимальная задержка	Сложность реализации
Микропакетная	Компромисс между batch и streaming	Баланс сложности и актуальности	Требует специальных инструментов

В рамках данной курсовой работы выбран подход пакетной обработки, так как данные о дефектах оборудования ТЭЦ не требуют анализа в реальном времени, а их обновление происходит с периодичностью, достаточной для ежедневной загрузки.

1.2. Анализ предметной области: дефекты оборудования ТЭЦ

Теплоэлектроцентр (ТЭЦ) – это сложный инженерный комплекс, предназначенный для комбинированной выработки электрической и тепловой энергии. Оборудование ТЭЦ работает в условиях высоких температур, давлений и механических нагрузок, что неизбежно приводит к возникновению дефектов.

На основе анализа предоставленных данных выявлены следующие типовые категории дефектов оборудования ТЭЦ:

- Течи, свечи, парение – наиболее распространенная категория дефектов (составляет около 93% от общего числа). Связана с разгерметизацией трубопроводов, арматуры и соединений.
- Подшипники – дефекты, связанные с износом подшипниковых узлов турбин, генераторов и насосного оборудования.
- Датчики – отказы контрольно-измерительных приборов и систем автоматизации.
- Сальники – дефекты уплотнительных устройств.
- Прочие дефекты – категория, требующая дополнительной детализации.

По результатам анализа данных выделены следующие ключевые показатели, необходимые для мониторинга, они представлены в таблице 2:

Таблица 2. Ключевые показатели для мониторинга

Показатель	Описание	Бизнес-ценность
Распределение дефектов по филиалам	Количество дефектов в разрезе филиалов организации	Выявление проблемных подразделений
Динамика по месяцам	Изменение количества дефектов во времени	Оценка эффективности ремонтных кампаний
Приоритетность дефектов	Распределение по уровням критичности (1-4)	Оптимизация планирования ремонтов
Дефектность по станциям	Сравнение ТЭЦ по количеству дефектов	Инвестиционные решения
Категории дефектов	Структура дефектов по типам оборудования	Техническая политика
Статус устранения	Доля устраненных/отложенных/отклоненных дефектов	Оценка эффективности

		ремонтной службы
--	--	------------------

Основные стейкхолдеры и их потребности в аналитике представлены в таблице 3.

Таблица 3. Основные стейкхолдеры и их потребности в аналитике

Стейкхолдер	Потребности
Руководство организации	Общая статистика, тренды, KPI
Главный инженер	Приоритетные дефекты, критичное оборудование
Начальник ремонтной службы	Статусы устранения, сроки выполнения
Технические специалисты	Детальная информация по каждому дефекту

1.3. Обзор современных технологий для построения ETL-конвейеров

Современный рынок предлагает широкий спектр инструментов для построения ETL-конвейеров. Их можно классифицировать по нескольким критериям: тип лицензии (открытые/проприетарные), способ развертывания (on-premise/cloud), целевая аудитория (enterprise/SMB).

Apache Airflow – это платформа с открытым исходным кодом для программного создания, планирования и мониторинга рабочих процессов (workflows). Airflow был разработан в компании Airbnb в 2014 году и с тех пор стал де-факто стандартом в области оркестрации ETL-процессов.

Ключевые особенности Airflow:

- DAG-ориентированность – рабочие процессы описываются в виде ориентированных ациклических графов (DAG) на языке Python.

- Богатая экосистема операторов – готовые операторы для работы с базами данных (PostgreSQL, MySQL, ClickHouse), облачными сервисами (AWS, GCP, Azure), API и файловыми системами.
- Web-интерфейс – визуальный мониторинг выполнения задач, просмотр логов, управление DAG.
- Масштабируемость – поддержка распределенного выполнения с использованием Celery Executor.
- Интеграция с Docker – легкое развертывание с использованием официального образа.

Prefect – современный аналог Airflow, разработанный с учетом недостатков последнего. Основное отличие – более простая модель состояний и встроенная поддержка потоковой обработки. Однако Prefect требует облачного аккаунта для использования всех функций (Prefect Cloud), что ограничивает его применение в on-premise средах.

Dagster – еще один подход к оркестрации данных с фокусом на тестируемость и повторное использование кода. Отличается более крутой кривой обучения по сравнению с Airflow.

Сравнительный анализ инструментов оркестрации представлены в таблице 4.

Таблица 4. Сравнительный анализ инструментов оркестрации

Критерий	Apache Airflow	Prefect	Dagster
Лицензия	Apache 2.0	Apache 2.0	Apache 2.0
Язык	Python	Python	Python
Web-интерфейс	Есть	Есть	Есть
Потоковая обработка	Нет	Есть	Есть
Сложность освоения	Средняя	Низкая	Высокая
Cloud-версия	Нет	Есть	Есть

Docker-образ	Официальный	Официальный	Официальный
Сообщество	Огромное	Растущее	Небольшое

Вывод: для данной курсовой работы выбран Apache Airflow как наиболее зрелый, документированный и широко распространенный инструмент с активным сообществом

1.4. Сравнительный анализ инструментов визуализации данных

Для визуализации результатов анализа дефектов ТЭЦ были рассмотрены следующие инструменты:

Apache Superset – это современная платформа с открытым исходным кодом для исследования и визуализации данных. Superset разработан в компании Airbnb (как и Airflow) и поддерживается Apache Software Foundation.

Ключевые возможности Superset:

- SQL Lab – мощный интерфейс для выполнения SQL-запросов с автодополнением и визуализацией результатов.
- No-code дашборды – создание интерактивных панелей без написания кода.
- Широкий выбор визуализаций – более 40 типов диаграмм (столбчатые, линейные, круговые, карты, sunburst и др.).
- Поддержка ClickHouse – нативная интеграция с ClickHouse через драйвер clickhouse-connect.
- Row-level security – разграничение доступа на уровне строк данных.
- Кэширование – встроенное кэширование запросов для ускорения работы.

Grafana изначально создавалась для мониторинга систем и визуализации метрик временных рядов. Основная область применения – анализ логов, метрик

и трассировки в real-time режиме. Grafana имеет высокую производительность, но уступает Superset в ad-hoc аналитике и работе с реляционными данными.

Redash – более легковесное решение, ориентированное в первую очередь на SQL-запросы и создание простых дашбордов. Redash прост в освоении, но имеет меньшее количество типов визуализаций по сравнению с Superset.

Сравнительный анализ инструментов визуализации представлен в таблице 5.

Таблица 5. Сравнительный анализ инструментов визуализации

Критерий	Apache Superset	Grafana	Redash
Лицензия	Apache 2.0	AGPL 3.0	BSD 2-Clause
SQL Lab	Есть	Нет	Есть
ClickHouse	Есть	Есть	Есть
Типов визуализаций	40+	30+	20+
No-code дашборды	Есть	Есть	Есть
Документация	Отличная	Отличная	Хорошая
Сообщество	Крупное	Огромное	Среднее
Docker-образ	Есть	Есть	Есть

Вывод: для данной курсовой работы выбран **Apache Superset** как наиболее функциональный и удобный инструмент для аналитической визуализации данных о дефектах ТЭЦ.

1.5. Обоснование выбора технологического стека

На основе проведенного анализа в таблице 6 ниже представлено итоговое обоснование выбора каждого компонента технологического стека.

Таблица 6. Итоговое обоснование выбора каждого компонента технологического стека

Компонент	Выбранная	Причины выбора
-----------	-----------	----------------

	технология	
Оркестрация ETL	Apache Airflow	• Стандарт индустрии • Огромное сообщество • Богатая экосистема операторов • Простота интеграции с ClickHouse и PostgreSQL • Официальный Docker-образ
Аналитическое хранилище	ClickHouse	• Молниеносная скорость агрегаций • Идеален для временных рядов и логов • Эффективное сжатие данных (10:1) • Нативная поддержка в Superset • Бесплатный (Apache 2.0)
Визуализация	Apache Superset	• Интегрирован с Airflow (общее происхождение) • Мощный SQL Lab • Нативная поддержка ClickHouse • Гибкая система безопасности • Активное развитие
Метаданные Airflow	PostgreSQL	• Официально рекомендуется Airflow • Поддержка конкурентности • Легкость резервного копирования • Надежность
Контейнеризация	Docker + Docker Compose	• Простота развертывания • Воспроизводимость среды

		• Переносимость между системами • Идемпотентность запуска
Обратный прокси + HTTPS	Nginx + Let's Encrypt	• Автоматическое получение сертификатов • Шифрование трафика • Поддержка Docker

Обоснование отказа представлен в таблице 7.

Таблица 7. Обоснование отказа

Компонент	Альтернатива	Почему не выбрано
Airflow → Prefect	Prefect 2.0	Требует облачный аккаунт (Prefect Cloud) для полноценной работы, что неприемлемо для on-premise развёртывания на ТЭЦ
ClickHouse → TimescaleDB	TimescaleDB (на базе PostgreSQL)	ClickHouse показывает в 5-10 раз более высокую производительность на агрегатных запросах, критичных для анализа дефектов
Superset → Grafana	Grafana	Grafana ориентирована на real-time мониторинг, а не на ad-hoc аналитику и исследование данных (SQL Lab отсутствует)
Docker → Kubernetes	Minikube / K3s	Избыточно для данного проекта; Docker Compose полностью покрывает потребности

Глава 2. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ ETL-КОНВЕЙЕРА

2.1. Общая архитектура решения

Разработанная архитектура ETL-конвейера для анализа дефектов ТЭЦ представляет собой совокупность взаимосвязанных компонентов, каждый из которых выполняет строго определенную функцию. Архитектура следует принципам микросервисного подхода: каждый компонент запускается в отдельном Docker-контейнере, что обеспечивает изоляцию, независимое масштабирование и простоту управления.

Общая схема архитектуры:

Компоненты архитектуры:

PostgreSQL – контейнер с базой данных для хранения метаданных Airflow (информация о DAG, задачах, запусках, соединениях, переменных).

ClickHouse – аналитическое хранилище данных, куда загружаются очищенные и трансформированные данные о дефектах. ClickHouse оптимизирован для выполнения агрегатных запросов.

Airflow Webserver – веб-сервер Airflow, предоставляющий пользовательский интерфейс для мониторинга и управления DAG. Доступен по порту 8080.

Airflow Scheduler – планировщик Airflow, который отвечает за запуск задач по расписанию. Взаимодействует с PostgreSQL для хранения состояния.

Superset – веб-приложение для визуализации данных, подключенное к ClickHouse. Предоставляет интерфейс для создания дашбордов и выполнения SQL-запросов.

Общая сеть (etl_network) – внутренняя сеть Docker, обеспечивающая изолированное взаимодействие между контейнерами.

Тома (Volumes) – механизм Docker для сохранения данных между перезапусками контейнеров.

Потоки данных в архитектуре представлены в таблице 8.

Таблица 8. Потоки данных в архитектуре

№	Откуда	Куда	Что передается	Периодичность
1	Внешний CSV	PostgreSQL	Исходные данные о дефектах	Разово (при загрузке)
2	PostgreSQL	ClickHouse	Очищенные и трансформированные данные	По расписанию DAG
3	ClickHouse	Superset	Данные для визуализации	В реальном времени
4	Superset	Пользователь	Визуализации, дашборды	По запросу
5	Пользователь	Airflow	Управление DAG	По запросу

2.2. Контейнеризация с использованием Docker и Docker

Compose

Контейнеризация является ключевым технологическим решением в данном проекте. Использование Docker позволяет упаковать каждый компонент системы вместе со всеми его зависимостями в изолированный контейнер, что обеспечивает воспроизводимость среды выполнения и упрощает развертывание на любых платформах, поддерживающих Docker.

Основные принципы контейнеризации, примененные в проекте:

- Разделение ответственности – каждый контейнер отвечает ровно за одну функцию.
- Использование официальных образов – для минимизации рисков безопасности и упрощения поддержки.
- Декларативность конфигурации – все настройки описываются в `docker-compose.yml`.
- Постоянство данных – критичные данные хранятся в Docker-томах.

Структура Docker-образов представлена в таблице 9.

Таблица 9. Структура Docker-образов

Сервис	Базовый образ	Кастомные слои
Airflow	apache/airflow:2.8.1	Установка clickhouse-driver, pandas
Superset	apache/superset:3.1.3	Установка clickhouse-connect
PostgreSQL	postgres:13	Без изменений
ClickHouse	clickhouse/clickhouse-server:23.8	Скрипт инициализации init.sql

2.3. Реализация ETL-процессов в Apache Airflow

В основе ETL-конвейера лежит Directed Acyclic Graph (DAG), написанный на языке Python с использованием Apache Airflow. DAG описывает последовательность задач, их зависимости и расписание выполнения.

Инструкция по созданию пользователя в Airflow:

```
docker exec -it etl-project-webserver-1 airflow users create \    --username  
admin \    --password admin \    --firstname Admin \    --lastname User \    --  
role Admin \    --email admin@example.com
```

2.4. Настройка ClickHouse как аналитического хранилища

ClickHouse – это колоночная система управления базами данных с открытым исходным кодом, предназначенная для аналитической обработки онлайн-запросов (OLAP). В рамках данного проекта ClickHouse выполняет роль целевого хранилища для загруженных и трансформированных данных о дефектах ТЭЦ.

Обоснование выбора ClickHouse представлен в таблице 10.

Таблица 10. Обоснование выбора ClickHouse

Характеристика	Значение для проекта
----------------	----------------------

Скорость агрегаций	Агрегатные запросы выполняются за миллисекунды на миллионах строк
Сжатие данных	Коэффициент сжатия 5-10:1, экономия дискового пространства
Поддержка SQL	Стандартный SQL, легкость освоения
Интеграция с Superset	Нативный драйвер clickhouse-connect

Структура базы данных (sql):

```
-- База данных для хранения информации о дефектах
CREATE DATABASE IF NOT EXISTS defects_db;
-- Основная таблица для детальных записей о дефектах
CREATE TABLE IF NOT EXISTS defects_db.defects_analysis (
    unique_number String,          -- Уникальный номер дефекта
    filial String,                 -- Филиал
    station String,               -- Станция/ТЭЦ
    priority Int32,                -- Приоритет (1-4)
    alias String,                  -- Статус (ELIMINATED, REJECTED, etc.)
    created_date Date,             -- Дата создания
    year_month String,             -- Год-месяц для агрегации
    load_timestamp DateTime DEFAULT now() -- Время загрузки
) ENGINE = MergeTree() ORDER BY (filial, created_date);
-- Агрегирующая таблица для дашбордов
CREATE TABLE IF NOT EXISTS defects_db.defects_dashboard (
    filial String,
    total_defects UInt32,
    update_date Date DEFAULT today()
) ENGINE = SummingMergeTree() ORDER BY filial;
```

2.5. Визуализация данных в Apache Superset

Apache Superset предоставляет мощные средства для визуализации данных. В рамках проекта были созданы следующие дашборды и графики.

Типы созданных визуализаций представлены в таблице 11.

Таблица 11. Типы созданных визуализаций

Название	Тип	Источник данных	Назначение
----------	-----	-----------------	------------

графика	визуализации		
Дефекты по филиалам	Bar Chart	defects_db.defects_analysis	Выявление проблемных подразделений
Динамика по месяцам	Line Chart	defects_db.defects_analysis	Анализ сезонности
Дефекты по приоритетам	Bar Chart	defects_db.defects_analysis	Оценка критичности
Дефекты по станциям	Horizontal Bar Chart	defects_db.defects_analysis	Ранжирование ТЭЦ
Категории дефектов	Pie Chart	defects_db.defects_analysis	Структура дефектов
Статусы с процентами	Pie Chart	defects_db.defects_analysis	Эффективность устранения

Шаг 1. Подключение базы данных ClickHouse:

- Открыть Settings → Database Connections → + Database
- Выбрать ClickHouse Connect
- Заполнить поля: Host: clickhouse, Port: 8123, Database: defects_db
- Test Connection → Connect

Шаг 2. Создание Dataset:

- Data → Datasets → + Dataset
- Выбрать базу данных и таблицу defects_analysis
- Create Dataset

Шаг 3. Создание графика "Дефекты по филиалам":

- Charts → + Chart → Bar Chart
- Dataset: defects_analysis
- Query: GROUP BY filial, METRICS COUNT(*)
- Run Query → Save

Шаг 4. Сборка дашборда:

- Dashboards → + Dashboard

- Название: "Анализ дефектов ТЭЦ"
- Edit Dashboard → Add Chart (добавить все графики)
- Настроить расположение → Save

Глава 3. ПРАКТИЧЕСКОЕ РАЗВЕРТЫВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

3.1. Развертывание проекта

Для развертывания проекта необходимо следовать приведенной ниже инструкции. Все команды выполняются в терминале (PowerShell или bash).

Требования к системе представлены в таблице 12.

Таблица 12. Требования к системе

Компонент	Минимальная версия	Рекомендуемая версия
Docker	24.0.0	25.0+
Docker Compose	2.20.0	2.24+
Оперативная память	4 ГБ	8 ГБ+
Место на диске	5 ГБ	10 ГБ

Ожидаемый результат выполнения команды `docker-compose ps`:

NAME	STATUS	PORTS
clickhouse	Up	0.0.0.0:8123->8123/tcp
etl-project-postgres-1	Up (healthy)	5432/tcp
etl-project-webserver-1	Up (healthy)	0.0.0.0:8080->8080/tcp
etl-project-scheduler-1	Up	8080/tcp
superset	Up	0.0.0.0:8088->8088/tcp

Доступ к сервисам представлен в таблице 13.

Таблица 13. Доступ к сервисам

Сервис	URL	Логин	Пароль
--------	-----	-------	--------

Airflow	http://localhost:8080	admin	admin
Superset	http://localhost:8088	admin	admin
ClickHouse	http://localhost:8123	default	-

3.2. Загрузка и обработка данных о дефектах

После успешного развертывания системы необходимо загрузить исходные данные о дефектах.

Структура исходного CSV-файла представлена в таблице 13.

Таблица 13. Структура исходного CSV-файла

Колонка	Тип	Описание
unique_number	VARCHAR	Уникальный идентификатор дефекта
filial	VARCHAR	Филиал
station	VARCHAR	Станция/ТЭЦ
priority	INTEGER	Приоритет (1-4)
alias	VARCHAR	Статус (ELIMINATED, REJECTED и др.)
created_date	DATE	Дата создания

Активация и запуск DAG в Airflow:

1. Открыть <http://localhost:8080> (admin/admin)
2. Найти DAG `tpp_defects_etl`
3. Переключить переключатель "Off" → "On"
4. Нажать кнопку "Trigger DAG" для ручного запуска
5. Наблюдать за выполнением задач в интерфейсе Airflow
6. Наблюдать за выполнением задач в интерфейсе Airflow

3.3. Создание дашбордов для анализа дефектов ТЭЦ

В Apache Superset были созданы следующие дашборды и визуализации.

Дашборд "Анализ дефектов ТЭЦ" включает:

1. Дефекты по филиалам (Bar Chart)

Вывод: наибольшее количество дефектов приходится на Невский филиал

2. Динамика дефектов по месяцам (Line Chart)

Вывод: статистика дефектов не имеет выраженной сезонности

3. Дефекты по приоритетам (Bar Chart)

Вывод: дефекты 2 и 3 приоритетов преобладают

4. Дефекты по станциям (Horizontal Bar Chart)

Вывод: ТЭЦ-68 требует повышенного внимания

5. Категории дефектов (Pie Chart)

Вывод: основная масса дефектов относится к категории "Течи, свечи, парение"

6. Статусы дефектов с процентами (Pie Chart)

Вывод: 93% дефектов устранены (ELIMINATED) — положительный показатель

3.4. Интерпретация полученных результатов

По результатам анализа данных были сделаны следующие выводы, которые представлены в таблице 14.

Таблица 14. Результат анализа данных

№	Показатель	Результат	Рекомендация
1	Распределение по филиалам	Невский филиал лидирует по числу дефектов	Провести внутренний аудит, усилить контроль
2	Сезонность дефектов	Выраженная сезонность отсутствует	Внедрить круглогодичный мониторинг
3	Приоритетность дефектов	Преобладают дефекты 2 и 3 приоритетов	Оптимизировать планирование ремонтов
4	Дефектность по станциям	ТЭЦ-68 требует внимания	Провести детальное обследование оборудования
5	Категории дефектов	Преобладают течи и парения	Усилить контроль герметичности

6	Статусы устранения	93% дефектов устранены	Положительная оценка работы ремонтной службы
---	--------------------	------------------------	---

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель – разработан и развернут ETL-конвейер для анализа дефектов оборудования ТЭЦ с использованием современных открытых технологий.

Основные результаты работы:

- Аналитический обзор – проведен анализ предметной области, выявлены основные типы дефектов оборудования ТЭЦ и требования к их анализу.
- Сравнительный анализ технологий – выполнено сравнение Apache Airflow с Prefect и Dagster, Apache Superset с Grafana и Redash, ClickHouse с TimescaleDB, обоснован выбор технологического стека.
- Архитектура решения – разработана микросервисная архитектура ETL-конвейера, включающая PostgreSQL (метаданные), ClickHouse (хранилище), Airflow (оркестрация) и Superset (визуализация).
- Контейнеризация – все компоненты упакованы в Docker-контейнеры, написан docker-compose.yml для оркестрации, настроены Docker-тома для постоянства данных.
- Реализация ETL – написан DAG в Apache Airflow, реализующий процессы извлечения, трансформации и загрузки данных, настроено ежедневное расписание выполнения.
- Визуализация – в Apache Superset созданы дашборды для визуализации результатов анализа: дефекты по филиалам, месяцам, приоритетам, станциям, категориям и статусам.
- Документация – составлена инструкция по развертыванию и использованию разработанного решения.

Практическая значимость работы заключается в том, что разработанное решение:

- может быть внедрено на предприятиях энергетического сектора;

- сокращает время на обработку данных о дефектах;
- обеспечивает наглядную визуализацию ключевых показателей;
- позволяет выявлять проблемные зоны и принимать обоснованные управленческие решения.

Перспективы развития проекта:

- Интеграция с SCADA-системами для получения данных в реальном времени;
- Внедрение предиктивной аналитики для прогнозирования отказов;
- Настройка автоматической рассылки отчетов по электронной почте;
- Развертывание в облачной инфраструктуре;
- Настройка HTTPS через Let's Encrypt для безопасного удаленного доступа.

СПИСОК ЛИТЕРАТУРЫ

1. Как ETL-процессы помогают анализировать большие данные // ЯндексПрактикум URL: <https://practicum.yandex.ru/blog/chto-takoe-etl/> (дата обращения: 09.06.2026).
2. ETL процессы извлечение преобразование и загрузка данных // KT-team URL: <https://www.kt-team.ru/blog/etl-data-integration-basics> (дата обращения: 09.06.2026).
3. Apache Airflow // Airflow.Apache URL: <https://airflow.apache.org/> (дата обращения: 09.06.2026).
4. Docker Compose // docs.docker.com URL: <https://docs.docker.com/compose/> (дата обращения: 09.06.2026).
5. Что такое ClickHouse? // ClickHouse URL: <https://clickhouse.com/docs/ru/intro> (дата обращения: 09.06.2026).
6. Apache Superset: Платформа для визуализации данных и аналитики бизнеса // superset-bi URL: <https://superset-bi.ru/> (дата обращения: 09.06.2026).
7. Что такое ETL-конвейер? Процесс, особенности и примеры // projectpro.io URL: <https://www.projectpro.io/article/how-to-build-etl-pipeline-example/526> (дата обращения: 09.06.2026).
8. Как запустить сайт с Docker, Nginx и Certbot: полный гайд // Хабр URL: <https://habr.com/ru/articles/937150/> (дата обращения: 09.06.2026).

ПРИЛОЖЕНИЕ А

A.1. docker-compose.yml

```
x-airflow-common:
  &airflow-common
  build: .
  environment:
    &airflow-env
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__WEBSERVER__DEFAULT_UI_TIMEZONE: 'Europe/Moscow'
  volumes:
    - ./dags:/opt/airflow/dags
    - ./data:/opt/airflow/data
    - ./logs:/opt/airflow/logs
  depends_on:
    postgres:
      condition: service_healthy
    clickhouse:
      condition: service_started
  env_file:
    - .env
  networks:
    - etl_network

services:
  postgres:
    image: postgres:13
    container_name: etl-project-postgres-1
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    networks:
      - etl_network
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 5s
      timeout: 5s
      retries: 10
      restart: always

  clickhouse:
    image: clickhouse/clickhouse-server:23.8
    container_name: clickhouse
    ports:
      - "8123:8123"
      - "9000:9000"
    volumes:
      - clickhouse-data:/var/lib/clickhouse
      - ./clickhouse:/docker-entrypoint-initdb.d
```

```

environment:
  CLICKHOUSE_DB: defects_db
  CLICKHOUSE_USER: default
  CLICKHOUSE_PASSWORD: ""
networks:
  - etl_network
restart: always

webserver:
  <<: *airflow-common
  container_name: etl-project-webserver-1
  command: >
    bash -c "
      airflow db migrate &&
      airflow users create --username admin --password admin --firstname Admin --lastname User --role
Admin --email admin@example.com || true &&
      airflow webserver
    "
  ports:
    - "8080:8080"
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 10
    start_period: 60s
  restart: always

scheduler:
  <<: *airflow-common
  container_name: etl-project-scheduler-1
  command: scheduler
  restart: always
  depends_on:
    webserver:
      condition: service_healthy

superset:
  image: apache/superset:3.1.3
  container_name: superset
  ports:
    - "8088:8088"
  environment:
    SUPERSET_SECRET_KEY: your-secret-key-change-this-in-production
    SUPERSET_CONFIG_PATH: /app/pythonpath/superset_config.py
  volumes:
    - ./superset_config.py:/app/pythonpath/superset_config.py
    - superset-data:/app/superset_home
  depends_on:
    clickhouse:
      condition: service_started
  networks:
    - etl_network
  restart: always
  command: >
    bash -c "

```

```

    superset db upgrade &&
    superset init &&
    superset fab create-admin --username admin --password admin --firstname Admin --lastname User
--email admin@example.com || true &&
    gunicorn --bind 0.0.0.0:8088 --workers 2 --timeout 60 --limit-request-line 0 --limit-request-field_size
0 'superset.app:create_app()'
"

healthcheck:
  test: ["CMD", "curl", "--fail", "http://localhost:8088/health"]
  interval: 30s
  timeout: 10s
  retries: 10
  start_period: 60s

networks:
  etl_network:
    driver: bridge

volumes:
  postgres-db-volume:
  clickhouse-data:
  superset-data:

```

A.2. Dockerfile (Airflow)

```

# Временно упрощённый Dockerfile
FROM apache/airflow:2.8.2-python3.11

USER root
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
    gcc \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

USER airflow

RUN pip install --no-cache-dir \
    pandas==2.0.3 \
    openpyxl==3.1.2 \
    sqlalchemy==1.4.49

```

A.3. Dockerfile.superset

```

FROM apache/superset:3.1.3

USER root

# Обновление pip и установка драйверов
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir \
    clickhouse-connect==0.6.8 \
    clickhouse-driver==0.2.6 \

```

```

psycpg2-binary \
redis

# Копирование конфига
COPY superset_config.py /app/pythonpath/superset_config.py

# Смена владельца
RUN chown -R superset:superset /app/pythonpath

USER superset

```

A.4. dags/defects_etl.py

```

from airflow.decorators import dag, task
from datetime import datetime, timedelta
import pandas as pd
import numpy as np
import os
from datetime import datetime as dt

from clickhouse_driver import Client

default_args = {
    'owner': 'analytics_team',
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=1),
    'start_date': datetime(2024, 1, 1),
}

schedule_interval = '0 8 * * *'

# Директория для временных файлов
TEMP_DIR = '/tmp/airflow_data'
os.makedirs(TEMP_DIR, exist_ok=True)

@dag(default_args=default_args, schedule_interval=schedule_interval, catchup=False, tags=['tezis',
'repair_analysis'])
def tec_breakdown_etl():
    @task
    def extract_from_excel():
        excel_path =
'/opt/airflow/data/select_d_unique_number_sv_filial_sv_station_sv_oe_e_full_name_d.xlsx'

        print(f"  Reading file: {excel_path}")
        df = pd.read_excel(excel_path, dtype=str, engine='openpyxl')
        print(f"✓ Extracted {len(df)} rows, {len(df.columns)} columns")

        # Сохраняем в Parquet
        output_path = os.path.join(TEMP_DIR, 'extracted_data.parquet')
        df.to_parquet(output_path, index=False)
        print(f"  Saved to: {output_path}")

    return output_path

```

```

@task
def transform_data(input_path):
    print(f"    Reading from: {input_path}")
    df = pd.read_parquet(input_path)
    print(f"    Initial shape: {df.shape}")

    column_mapping = {
        'unique_number': 'defect_id',
        'filial': 'branch',
        'station': 'station',
        'oe': 'department',
        'full_name': 'equipment_name',
        'description': 'defect_description',
        'created_date': 'created_date_raw',
        'alias': 'status',
        'fio': 'responsible_person',
        'priority': 'priority_raw',
        'Плановый_срок_устранения_дефекта': 'planned_deadline_raw',
        'Фактический_срок_устранения_дефекта': 'actual_fix_date_raw'
    }

    # Переименовываем только существующие колонки
    existing_mapping = {k: v for k, v in column_mapping.items() if k in df.columns}
    df = df.rename(columns=existing_mapping)

    # Выбираем нужные колонки
    cols_to_keep = ['defect_id', 'branch', 'station', 'department', 'equipment_name',
                    'defect_description', 'status', 'responsible_person', 'priority_raw',
                    'created_date_raw', 'planned_deadline_raw', 'actual_fix_date_raw']
    df = df[[col for col in cols_to_keep if col in df.columns]]

    df['created_date'] = pd.to_datetime(df['created_date_raw'], errors='coerce', dayfirst=True)
    df['planned_deadline'] = pd.to_datetime(df['planned_deadline_raw'], errors='coerce', dayfirst=True)
    df['actual_fix_date'] = pd.to_datetime(df['actual_fix_date_raw'], errors='coerce', dayfirst=True)

    df['priority'] = pd.to_numeric(df['priority_raw'], errors='coerce')
    df['priority'] = df['priority'].fillna(0).astype(int)

    # Расчет метрик
    df['fix_time_days'] = (df['actual_fix_date'] - df['created_date']).dt.days
    df['overdue_days'] = (df['actual_fix_date'] - df['planned_deadline']).dt.days
    df['is_overdue'] = np.where(df['overdue_days'] > 0, 1, 0)
    df['is_plan_met'] = np.where(df['actual_fix_date'] <= df['planned_deadline'], 1, 0)

    # Категоризация дефектов
    df['defect_description'] = df['defect_description'].fillna("")
    df['defect_category'] = 'other'
    df.loc[df['defect_description'].str.contains('te|svisch', case=False, na=False), 'defect_category'] =
    'leak'
    df.loc[df['defect_description'].str.contains('salnik', case=False, na=False), 'defect_category'] = 'seal'
    df.loc[df['defect_description'].str.contains('zadvizhk|ventil|armatur', case=False,
                                                  na=False), 'defect_category'] = 'valve'
    df.loc[df['defect_description'].str.contains('elektric|kabel|osvescheni', case=False,
                                                  na=False), 'defect_category'] = 'electrical'

```



```

# Фильтрация
df = df.dropna(subset=['created_date'])
df_eliminated = df[df['status'] == 'ELIMINATED'].copy()

# Очистка от выбросов
df_eliminated['fix_time_days'] = df_eliminated['fix_time_days'].fillna(0)
df_eliminated['overdue_days'] = df_eliminated['overdue_days'].fillna(0)
df_eliminated.loc[df_eliminated['fix_time_days'] < 0, 'fix_time_days'] = 0
df_eliminated.loc[df_eliminated['fix_time_days'] > 1000, 'fix_time_days'] = 0
df_eliminated.loc[df_eliminated['overdue_days'] < -1000, 'overdue_days'] = 0

df_eliminated['fix_time_days'] = df_eliminated['fix_time_days'].astype(int)
df_eliminated['overdue_days'] = df_eliminated['overdue_days'].astype(int)
df_eliminated['is_overdue'] = df_eliminated['is_overdue'].astype(int)
df_eliminated['is_plan_met'] = df_eliminated['is_plan_met'].astype(int)

string_cols = ['responsible_person', 'branch', 'station', 'department', 'equipment_name']
for col in string_cols:
    df_eliminated[col] = df_eliminated[col].fillna("")

df_eliminated = df_eliminated.drop(
    columns=['created_date_raw', 'planned_deadline_raw', 'actual_fix_date_raw', 'priority_raw'])

print(f"✓ Transformation complete. {len(df_eliminated)} records to load")

output_path = os.path.join(TEMP_DIR, 'transformed_data.parquet')
df_eliminated.to_parquet(output_path, index=False)
print(f"  Saved transformed data to: {output_path}")

return output_path

@task
def load_data(input_path):
    """Загрузка данных в ClickHouse"""
    print(f"  Reading transformed data from: {input_path}")
    df = pd.read_parquet(input_path)
    print(f"  Loading {len(df)} records to ClickHouse")

    # Подключаемся к ClickHouse
    client = Client(host='clickhouse', port=9000, user='default', password='')
    print(f"✓ Connected to ClickHouse")

    # Создаем таблицу с Nullable типами для дат
    client.execute("""
        CREATE TABLE IF NOT EXISTS defect_analysis (
            defect_id String,
            branch String,
            station String,
            department String,
            equipment_name String,
            defect_description String,
            status String,
            responsible_person String,
            priority Int32,
            created_date Nullable(Date),
            planned_deadline Nullable(Date),

```

```

        actual_fix_date Nullable(Date),
        fix_time_days Int32,
        overdue_days Int32,
        is_overdue Int32,
        is_plan_met Int32,
        defect_category String
    ) ENGINE = MergeTree()
    ORDER BY (branch, created_date)
""")
print("✔ Table created/verified")

columns_order = [
    'defect_id', 'branch', 'station', 'department', 'equipment_name',
    'defect_description', 'status', 'responsible_person', 'priority',
    'created_date', 'planned_deadline', 'actual_fix_date', 'fix_time_days',
    'overdue_days', 'is_overdue', 'is_plan_met', 'defect_category'
]

for date_col in ['created_date', 'planned_deadline', 'actual_fix_date']:
    if date_col in df.columns:
        df[date_col] = pd.to_datetime(df[date_col], errors='coerce')
        # Преобразуем в date объекты или None
        df[date_col] = df[date_col].apply(lambda x: x.date() if pd.notna(x) else None)

# Заполняем числовые NaN нулями
numeric_cols = ['priority', 'fix_time_days', 'overdue_days', 'is_overdue', 'is_plan_met']
for col in numeric_cols:
    if col in df.columns:
        df[col] = df[col].fillna(0).astype(int)

# Заполняем строковые NaN пустыми строками
string_cols = ['defect_id', 'branch', 'station', 'department', 'equipment_name',
               'defect_description', 'status', 'responsible_person', 'defect_category']
for col in string_cols:
    if col in df.columns:
        df[col] = df[col].fillna("").astype(str)

# Создаем список кортежей для вставки
data_to_insert = []
problematic_records = 0

for _, row in df.iterrows():
    row_data = []
    valid = True
    for col in columns_order:
        value = row[col]

        # Проверка валидности дат
        if col in ['created_date', 'planned_deadline', 'actual_fix_date']:
            if value is not None:
                try:
                    # Проверяем, что это валидный date объект
                    if hasattr(value, 'year'):
                        _ = value.year
                    else:

```

```

        value = None
    except:
        value = None

    row_data.append(value)

    data_to_insert.append(tuple(row_data))

print(f"✔ Prepared {len(data_to_insert)} records for insertion")

if len(data_to_insert) == 0:
    print("  No valid data to insert")
    return "No data loaded"

# Вставляем данные пакетами
batch_size = 500
total_inserted = 0

for i in range(0, len(data_to_insert), batch_size):
    batch = data_to_insert[i:i + batch_size]
    try:
        insert_query = f"""
            INSERT INTO defect_analysis ({', '.join(columns_order)})
            VALUES
            """
        client.execute(insert_query, batch)
        total_inserted += len(batch)
        print(f"✔ Inserted batch {i // batch_size + 1}: {len(batch)} records")
    except Exception as e:
        print(f"✗ Error inserting batch {i // batch_size + 1}: {e}")
        # Пробуем вставить по одной записи из проблемного батча
        print("Attempting to insert records one by one...")
        for record in batch:
            try:
                client.execute(insert_query, [record])
                total_inserted += 1
            except Exception as record_error:
                problematic_records += 1
                print(
                    f"Failed to insert record: {record[:5] if len(record) > 5 else record}... Error:
{record_error}")
        print(f"Successfully inserted {total_inserted} records after fallback")

    print(f"✔ Successfully inserted {total_inserted} records (skipped {problematic_records} problematic
records)")

# Проверяем количество записей
count = client.execute("SELECT COUNT(*) FROM defect_analysis")[0][0]
print(f"✔ Total records in table: {count}")

# Очищаем временные файлы
try:
    os.remove(input_path)
    extracted_file_path = os.path.join(TEMP_DIR, 'extracted_data.parquet')
    if os.path.exists(extracted_file_path):
        os.remove(extracted_file_path)

```

```

        print("    Temporary files cleaned up")
    except Exception as e:
        print(f"    Could not clean temp files: {e}")

    return f"Loaded {total_inserted} records, skipped {problematic_records}"

# Определяем DAG
extracted_file = extract_from_excel()
transformed_file = transform_data(extracted_file)
load_data(transformed_file)

# Создаем экземпляр DAG
tec_breakdown_etl = tec_breakdown_etl()

```

A.5. superset_config.py

```

import os

SECRET_KEY = os.environ.get('SUPERSET_SECRET_KEY', 'your-secret-key-change-this-in-production')

# Используем SQLite для метаданных (проще для начала)
SQLALCHEMY_DATABASE_URI = 'sqlite:///app/superset_home/superset.db'

# Отключаем загрузку примеров
SUPERSET_LOAD_EXAMPLES = False

# Настройки для ClickHouse
FEATURE_FLAGS = {
    "ENABLE_TEMPLATE_PROCESSING": True,
}

# Таймзона
DEFAULT_TIMEZONE = 'Europe/Moscow'

# Лимит строк
ROW_LIMIT = 10000

# Кэширование
CACHE_CONFIG = {
    'CACHE_TYPE': 'SimpleCache',
    'CACHE_DEFAULT_TIMEOUT': 300,
}

# Экспорт данных
CSV_EXPORT = {
    "encoding": "utf-8",
    "sep": ",",
    "null": "",
    "quoting": "minimal"
}
EXCEL_EXPORT = True

```

A.6. clickhouse/init.sql

```
CREATE DATABASE IF NOT EXISTS defects_db;

CREATE TABLE IF NOT EXISTS defects_db.defects_agg (
    turbine_id String,
    defect_type String,
    total_events UInt32,
    avg_criticality Float32,
    max_temp_delta Float32,
    dt Date
) ENGINE = MergeTree()
ORDER BY (turbine_id, dt);
```